Reproducing and Extending #GNN

Hashing-Accelerated Graph Neural Networks for Link Prediction

Wu, Li, Luo, Nejdl — WWW 2021

V Sai Siddhartha , Roll No: 22B0954
V Gnana Mahesh , Roll No: 22B0949
V Madhav , Roll No: 22B0993

May 10, 2026

CS 768

Outline

Link Prediction

Why Hashing Beats Standard GNNs Here

The #GNN Algorithm

Our Modifications

Experimental Setup

Results

Discussion

Conclusion

Link Prediction

The Link-Prediction Problem

Setup. A network $G = (V, E)$ where some edges are observed and some are hidden.

Question. Which currently-unconnected pairs of nodes are most likely to be connected?

Examples.

- Friend recommendation (Twitter, Facebook)
- Product recommendation (Amazon)
- Protein–protein interaction prediction
- Knowledge-graph completion

Attributed networks.
Each node also carries a feature set
(e.g. a Twitter user's hashtags).

Evaluation.
Hide some edges, sample non-edges, ask the model to rank.
Metric: **AUC** $=$ Pr[positive ranks above negative].
$1.0 =$ perfect, $0.5 =$ random.

Why Hashing Beats Standard GNNs Here

Standard GNNs vs. #GNN

Standard GNN
(GCN, GraphSAGE, GAT, SEAL, P-GNN. . .)

- Learned weight matrices
- Backprop, gradients, GPU
- Hours to train on large graphs
- Sensitive to hyperparameters
- Adding a new node $\rightarrow$ retrain

#GNN (this paper)

- Random MinHash functions instead of learned weights
- No training, no gradients
- Single forward pass, CPU-only
- 100–10,000$\times$ faster than learned GNNs
- Inductive by construction

Trade-off: small AUC gap on some datasets, but the speed gain is enormous and AUC stays competitive (often within 1–2%).

The Key Idea

Why MinHash works for graph similarity
For two sets A, B and a uniformly random hash function π,

$$\Pr\left[\arg\min_{a\in A}\pi(a) = \arg\min_{b\in B}\pi(b)\right] \;=\; \frac{|A\cap B|}{|A\cup B|} \;=\; \mathrm{Jaccard}(A,B).$$

Implication. If we "MinHash" the neighborhood-feature set of every node K times in parallel, then the **Hamming similarity** of the resulting two length-K codes is an unbiased estimator of the Jaccard similarity of their neighborhoods.

No learning required. The randomness in π does the "feature embedding" work.

The #GNN Algorithm

Setup and Hash Family

Input. Attributed graph $G = (V, E, X)$ where each $X_v \subseteq \mathcal{A}$ is a node's feature set; iterations T, embedding dim K.

Per iteration t, per dimension k, sample three independent universal hash functions of the form

$$\pi(i) \;=\; (a\,i + b) \bmod c, \qquad c \text{ prime},\; c \geq |\mathcal{A}|.$$

- π_1: hashes *self*-features in Phase 2.
- π_2: hashes neighbor messages in Phase 2.
- π_3: produces the diffused message in Phase 1.

Initial state. $x_v^{(0)} = X_v$ (the active attribute IDs).

Two-Phase Update

Phase 1 — Diffusion. Per node v, compute a single per-iteration message

$$m_v^{(t,k)} \;=\; \arg\min_{a \in x_v^{(t-1)}} \pi_3(a).$$

Phase 2 — Aggregation. Each new coordinate is the `argmin` over the union of hashed self-features and hashed neighbor messages:

$$x_v^{(t)}[k] \;=\; \arg\min \left\{\pi_1(a) : a \in x_v^{(t-1)}\right\} \cup \left\{\pi_2(m_u^{(t,k)}) : u \in \mathcal{N}(v)\right\}.$$

After T iterations, $x_v^{(T)}$ is the length-K embedding.

Similarity. $\mathrm{Sim}(u,v) = \frac{1}{K}\sum_k \mathbf{1}[\,x_u^{(T)}[k] = x_v^{(T)}[k]\,]$ (Hamming similarity).

Complexity. $\mathcal{O}(T \cdot K \cdot |V| \cdot (\bar{d} + |X_v|))$ — linear in edges. CPU only.

Our Modifications

Three Modifications We Test

Mod.	Idea	Expectation
A. JK concat	Concatenate embeddings from *all* iterations $1 \dots T$ instead of using only iteration T.	help
B. α	Make the self-vs-neighbor balance an explicit knob (probability α of taking from self only).	help
C. cap	Randomly subsample each neighborhood to at most 30 nodes before aggregating.	fail

Each modification is independently toggleable; we also test the "all three" combination.

Modification A: Jumping-Knowledge Concat

Motivation. Different node pairs need different proximity scales:

- Short-range pair $\rightarrow$ iteration 1 (one-hop overlap)
- Long-range pair $\rightarrow$ iteration T

The original algorithm *discards* iterations $1, \ldots, T-1$.

Change. Output the concatenation $[x_v^{(1)}; x_v^{(2)}; \ldots; x_v^{(T)}]$ of length $T \cdot K$.

Cost. $T\times$ larger embedding (still tiny), no extra iterations.

Inspired by Xu et al., *Jumping Knowledge Networks*, ICML 2018.

Modification B: Self-Influence α

Motivation. In Phase 2 the original algorithm merges self and neighbor candidates into one set before `argmin`.
The implicit balance depends on $|X_v|$ vs. $|\mathcal{N}(v)|$:

- Dense graphs: neighbors dominate.
- Sparse graphs: self dominates.

Neither is necessarily optimal.

Change. Per dimension k, with probability α take the `argmin` over self only; otherwise over neighbors only.

Modification C: Neighborhood Cap (designed to fail)

Motivation (naïve). High-degree hubs contribute large neighbor sets; perhaps random subsampling to 30 denoises them.

Why it should fail (theoretical).

- MinHash already *is* uniform sampling — collision probability equals the Jaccard of the *full* sets.
- Capping inputs $\rightarrow$ we now estimate the Jaccard of two random 30-subsets, an estimator with higher variance and downward bias.
- No runtime saving either: per-node cost is $\mathcal{O}(K + \deg(v))$ with $K = 200$, so the K term dominates.

Predicted outcome. AUC drops, runtime unchanged or worse.

Experimental Setup

Setup

Datasets. Three from the paper, used at full size.

	Nodes	Edges	Attributes	Avg. deg. (train, 80%)
Twitter	2,511	37,154	9,073	23.67
Facebook	4,039	88,234	1,403	34.95
Blog	5,196	171,743	8,189	52.88

Hyperparameters. Paper recommendations: $K = 200$; $T = 3$ for all datasets.

Train ratio. 0.8 (predefined split shipped with repo). The "Avg. deg." above is for the *training graph* the algorithm actually sees.

AUC protocol. Same as official `lp_evaluation.m`: 10,000 random non-edges + 10,000 random test edges, paired.

Hardware. CPU only.

Results

Main Results: AUC at Training Ratio 0.8

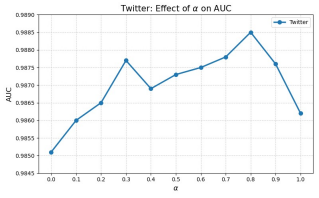
Configuration	Twitter	Facebook	Blog
Official C++ (paper)	0.9864	0.9823	0.7185
Original (our reimpl.)	0.9862	0.9842	0.7094
+ JK concat	0.9875	0.9811	**0.7243**
+ $\alpha = 0.7$	0.9878	**0.9849**	0.7234
+ neigh-cap = 30	0.9859	0.9826	0.5921
all three	**0.9887**	0.9828	0.6502

Our reimpl. matches the official C++ within 1 pp on every dataset ($\leq$ 0.91 pp, on Blog). $\alpha = 0.7$ is the only modification that improves AUC on *every* dataset.

Embedding Time (seconds)

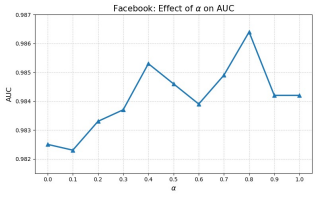
Configuration	Twitter	Facebook	Blog
Original (our reimpl.)	20.72	36.12	69.11
+ JK concat	22.09	40.46	71.77
+ $\alpha = 0.7$	21.08	36.04	59.26
+ neigh-cap = 30	26.87	47.39	73.93
all three	24.58	43.40	67.99

Sensitivity to α — Three Datasets



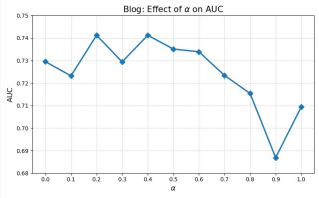
Twitter (deg 23.67,

peak ≈ 0.8)



Facebook (deg 34.95,

peak ≈ 0.8)



Blog (deg 52.88,

peak ≈ 0.2–0.4)

All curves at $T = 3$, $K = 200$. Degrees are average training-graph degree (training ratio 0.8).

The α Curves Tell a Story

All three datasets use $T = 3$, so the differences come from the graphs themselves. They line up by avg. training-graph degree:

- **Twitter** (deg 23.67, AUC ≈ 0.99): inverted-U, peak near $\alpha \approx 0.8$. Self-features (hashtags) carry strong signal.
- **Facebook** (deg 34.95, AUC ≈ 0.98): inverted-U, peak near $\alpha \approx 0.8$. Larger neighborhood, but small attribute set (1,403 tags) keeps each match informative.
- **Blog** (deg 52.88, AUC ≈ 0.72): noisy curve, peak at low α (0.2–0.4). Denser graph $\Rightarrow$ neighbors lead.

The pattern
Optimal α shifts toward 0 (neighbors-only) as average training-graph degree grows — denser graphs let neighbor messages aggregate richer signal that self-features cannot match.

Discussion

Sanity Check: Official vs. Our Reimplementation

Across all three datasets the official C++ AUC and our Python "original" AUC differ by:

Dataset	$\lvert\Delta\mathrm{AUC}\rvert$
Twitter	0.02 pp
Facebook	0.19 pp
Blog	0.91 pp

All within the noise expected from independent random hash-function draws (<1 pp).
Our reimplementation is faithful $\Rightarrow$ modification rows are fair tests.

Modification A: JK Concat (mixed)

Result.

- Twitter: $+0.13$ pp (helps slightly)
- Blog: $+1.49$ pp (helps most)
- Facebook: -0.31 pp (slightly hurts)

Pattern. JK helps on **underfit** datasets where iteration $T=3$ has not yet captured all structure (Blog at AUC ~ 0.71 has lots of headroom). It hurts on **saturated** datasets where iteration $T=3$ already captures most of the signal — earlier iterations then introduce noise into the Hamming similarity.

What predicts saturation? Combination of headroom and attribute richness. Facebook (AUC ≈ 0.98, only 1,403 attrs) has little room to improve and saturates at $T=3$, so JK noise dominates. Blog (AUC ≈ 0.71) has plenty of room.

Modification B: α (works)

Result at fixed $\alpha = 0.7$ (improves all three datasets):

- Twitter: $+0.16$ pp
- Facebook: $+0.07$ pp
- Blog: $+1.40$ pp (largest single gain)

The α-sweep reveals the catch: the optimal α is dataset-dependent, ranging from 0.2–0.4 (Blog) up to $\approx$0.8 (Twitter, Facebook).

Even non-optimal $\alpha = 0.7$ **still beats the original algorithm** on every dataset — the explicit knob is itself an improvement over the implicit merge.

Per-graph tuning would do better, at the cost of one more hyperparameter.

Modification C: Neighborhood Cap (fails)

Result.

- Twitter: -0.03 pp (cap 30 $>$ avg. deg. 23.67)
- Facebook: -0.16 pp (cap just below avg. deg. 34.95)
- Blog: -11.73 pp (catastrophic)

Damage scales with how much we throw away. Blog has the highest training-graph degree (52.88) and was already underfit (AUC ≈ 0.71); a cap of 30 throws away on average more than 40% of every neighborhood at every iteration.

Conclusion

Conclusion

- Reproduced #GNN within 1 pp AUC of the official C++ binary on all three datasets.
- α **(B)**: most reliable modification — helped on every dataset. Optimal α shifts toward 0 as average training-graph degree grows.
- **JK concat (A)**: mixed — helps when underfit (Blog) or unsaturated (Twitter), slightly hurts when saturated (Facebook).
- **Neighborhood cap (C)**: fails for the predicted theoretical reason; magnitude scales with neighbors discarded (-11.7 pp on Blog).

Takeaway
Hashing-based GNNs reward modifications that change *what information is exposed to the similarity computation*, and punish those that interfere with the underlying statistical estimator.

Work Distribution

Member	Contributions
V Sai Siddhartha	Implemented the original `hash_gnn` algorithm in Python; designed and implemented the self-influence parameter α ran the α-sweep across all datasets.
V Gnana Mahesh	Set up the official repo and patched/compiled the C++ binary; built the Python data-loading utilities for the official `.mat` / `.adjlist` format; designed and implemented Jumping-Knowledge concatenation.
V Madhav	Designed the random neighborhood cap and its theoretical failure analysis; generated α plots and result tables; prepared the report and LaTeX slides.

Thank you.

Questions?